

第24讲实验设计

一个神经元的 Backpropagation: 手算、数值差分与 Autograd 三重对照

核心概念: 实验设计

本讲把“节点级 backward”组装成一个真正的神经元反向传播。目标不是背公式，而是理解每个 Weight、Bias、Input 为什么会得到对应的梯度，以及这些梯度的 Shape 为什么与原变量一致。



建议学习方式: 先跟着一个 3 输入神经元手算 -> 再看向量公式 -> 最后运行 Gradient Check 与 PyTorch Autograd 对照。

一、实验目标

构造一个 3 输入、1 输出的人工神经元并完成完整 Forward。

按 Chain Rule 手工计算 dL/dw 、 dL/db 、 dL/dx 。

逐元素用 central difference 做 Gradient Check。

观察每个 Weight 梯度与对应输入 x_i 的直接比例关系。

做一次最小参数更新，验证小步沿负梯度方向可以降低当前 Loss。

构造 ReLU 关闭案例，观察 Loss 非零但参数梯度为0。

用 PyTorch Autograd 对照手工结果。

做一个 Batch 梯度聚合预览，为第25讲矩阵 Backprop 做准备。

二、实验问题与假设

实验问题	可检验假设
Q1 手工 backward 是否正确?	数值差分与手工梯度应在浮点误差范围内一致
Q2 Weight 梯度为什么不同?	同一 dL/dz 被不同 x_i 缩放, 因此 dL/dw_i 与 x_i 成正比
Q3 一小步负梯度更新能否降 Loss?	在足够小 $1r$ 下, 当前 Loss 应下降
Q4 ReLU 关闭会怎样?	$da/dz=0$ 时, 前方参数梯度应被截断
Q5 Autograd 做了什么?	PyTorch 应给出与手工公式相同的梯度
Q6 Batch 共享参数怎样聚合?	每个样本对共享参数的贡献按 reduction 规则求和/平均

三、固定实验设置

变量	数值
x	[2,-1,3]
w	[0.5,-2,1]
b	0.2
target	5.0
Activation	ReLU
Loss	$(a - \text{target})^2$
Gradient Check eps	1e-5
一步更新 learning rate	0.02

Forward 预期:

$z=6.2 \rightarrow a=6.2 \rightarrow \text{error}=1.2 \rightarrow \text{Loss}=1.44$

四、实验1：手工 Forward + Backward

按以下顺序计算并记录：



预期梯度：

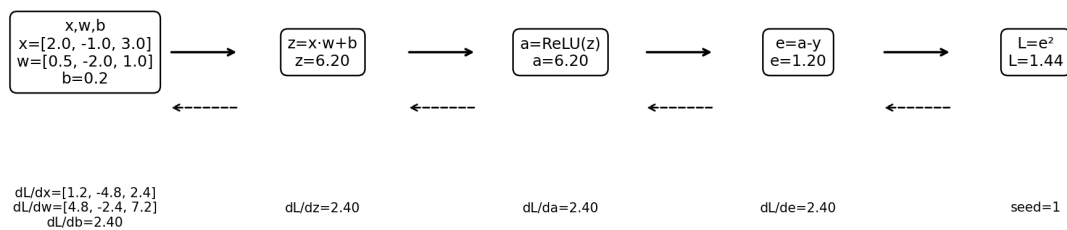
$$dL/dz = 2.4$$

$$dL/dw = [4.8, -2.4, 7.2]$$

$$dL/db = 2.4$$

$$dL/dx = [1.2, -4.8, 2.4]$$

单神经元：Forward 算数值，Backward 反向传梯度



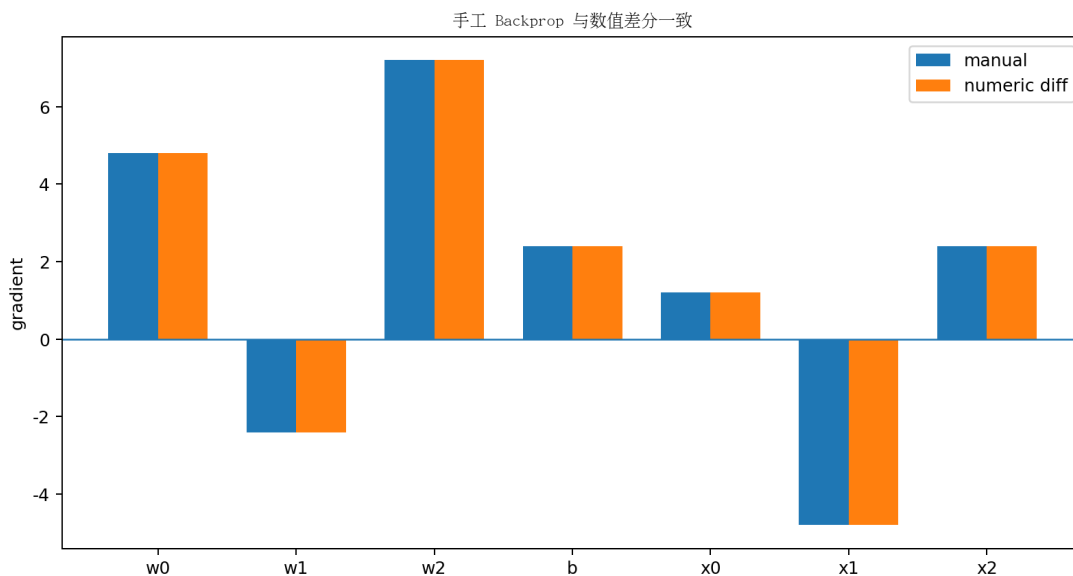
五、实验2：逐元素 Gradient Check

对任意一个变量元素 q :

$$g_{\text{numeric}} = [L(q+\text{eps}) - L(q-\text{eps})] / (2\text{eps})$$

分别检查 $w_0, w_1, w_2, b, x_0, x_1, x_2$ 。通过标准:

$$|g_{\text{manual}} - g_{\text{numeric}}| < 1e-6$$



数学直觉：为什么要逐元素

向量梯度其实就是“每个元素各自一个偏导数组成的向量”。逐元素检查可以直接证明 Shape 中每个位置的梯度都正确。

六、实验3：Weight 梯度与输入值

保持同一个 $dL/dz=2.4$ ，检查：

i	x_i	dL/dw_i
0	2	4.8
1	-1	-2.4
2	3	7.2

验证公式：

$$dL/dw_i = (dL/dz) x_i$$

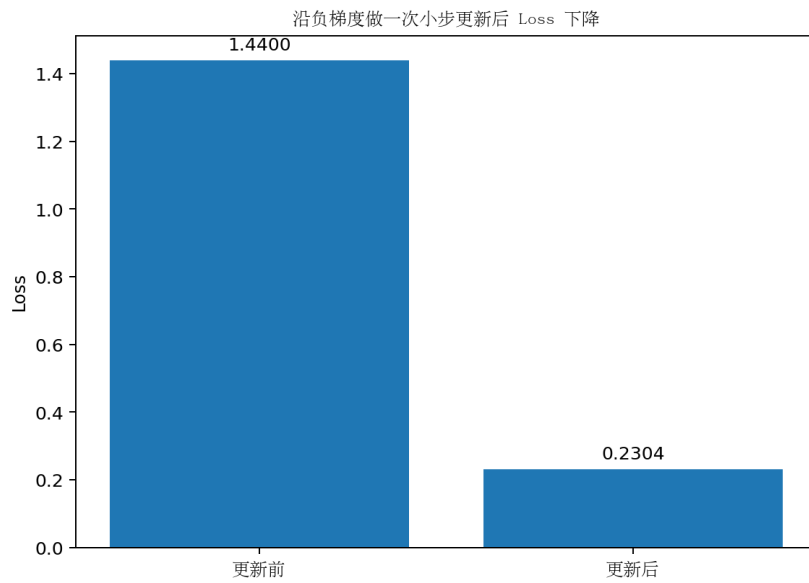
科研连接：科研意义

以后分析梯度时，不要只看 Weight 本身。当前激活/输入幅值、上游梯度、Normalization 等都会共同决定具体梯度大小。

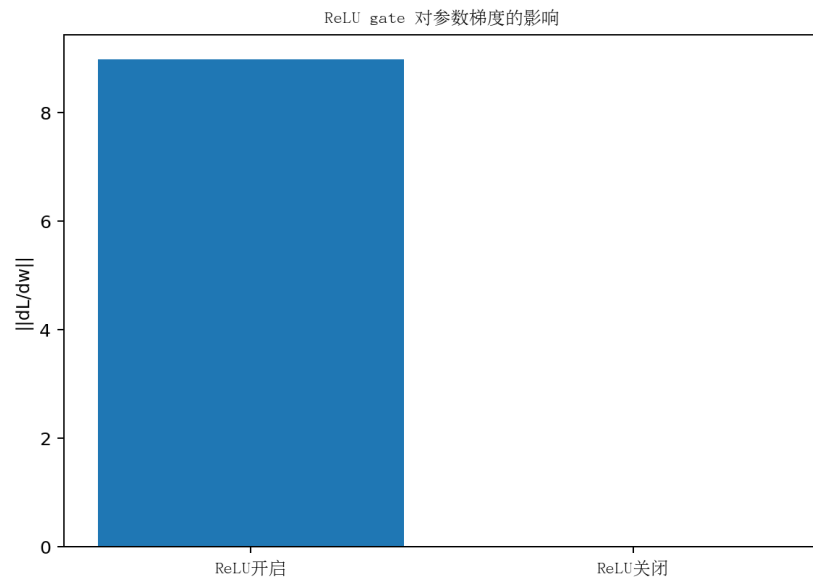
七、实验4：一次参数更新与 ReLU gate

先用 $1r=0.02$ 做一次更新。预期：

Loss 1.44 -> 0.2304



再构造 $z < 0$ 的案例。预期：ReLU 局部导数=0，因此 dL/dw 与 dL/db 为0。



八、实验5：PyTorch Autograd 对照

仅作为验证。核心代码：

```
xt = torch.tensor(x, requires_grad=True)
wt = torch.tensor(w, requires_grad=True)
bt = torch.tensor(b, requires_grad=True)

z = (xt * wt).sum() + bt
a = torch.relu(z)
loss = (a - target) ** 2
loss.backward()
```

应观察：

项	手工	Autograd
w.grad	[4.8,-2.4,7.2]	一致
b.grad	2.4	一致
x.grad	[1.2,-4.8,2.4]	一致

九、实验6：Batch 聚合预览

构造两个样本共享同一套 w, b 。Loss 使用 mean reduction。记录每个样本 z 、ReLU gate、逐样本 loss、逐样本 dL/dz ，以及最终聚合后的 dL/dw 、 dL/db 。

核心概念：预期现象

不同样本可以对同一个 Weight 贡献不同方向、不同大小，甚至0梯度；共享参数最终看到的是这些贡献按 loss reduction 规则聚合后的结果。

十、实验记录模板

检查项	记录内容
Forward	z / a / error / Loss
Manual backward	dL/dz / dL/dw / dL/db / dL/dx
Gradient Check	manual / numeric / abs error
Update	更新前后 $w, b, Loss$
ReLU off	z / Loss / grad norm
Autograd	$w.grad$ / $b.grad$ / $x.grad$
Batch preview	per-sample contribution / aggregated grad

本讲代码会自动把实际结果写入 TXT，便于以后复现实验和核对。

十一、结果解释与科研习惯

不要只记录“代码能跑”，必须同时记录输入、输出、Shape、梯度和关键超参数。

手工公式、数值差分、Autograd 三者一致时，才有较强证据说明实现正确。

如果 Gradient Check 失败，先查 Shape / reduction / activation 分支 / 符号 / 是否遗漏多路径累加。

不要根据单个样本的梯度直接做全局因果解释；它只是当前参数和当前样本附近的一阶局部敏感度。

从第25讲开始，要养成打印中间 Tensor Shape 和 Gradient Shape 的习惯。